

HEAPS AND AVL TREES

- Introduction to Heaps
- Heap Sort
- **Priority Queue**
- AVL Trees
 - Terminologies
 - Basic Operations
 - Rotation
 - Insertion
 - Deletion

Priority Queues - Operations

- Priority Queue using an array is one of the methods to implement a queue.
- Insertion of elements in the priority queue is similar to the **ordinary queue**
- Deletion of the elements is done according to the **priority**
- The elements in the priority queue have some priority. The priority of the element is used to determine the order in which the elements will be processed.

Priority Queues – Enqueue Operations

```
Enqueue(item)
{
    if(n==MAX_SIZE-1) // Checking Full Condition
    {
        Display("Queue is full");
        return;
    }
    queue[n++]=item; //
}
```

Priority Queues – Dequeue Operations

Dequeue()

```
{
    int item;
    if(n== -1) // Checking empty Condition
    {
        Display("Queue is Empty");
        return;
    }
    int i, max =0; // Find the maximum Priority
    for (i = 1; i < n; i++)
    {
        if (queue[max] < queue[i])
            max=i;
    }
    item = queue[max];
    // replace the max with the last element
    queue[max] = queue[n - 1];
    n=n-1;
    return item;
}
```

Node Structure

Struct Node

{

int priority;

int info;

struct Node *link;

};

PRIORITY	INFO	LINK
----------	------	------

NODE

Priority Queues – Using Linked List

void insert(**int** item,**int** priority) // **Insert a new element into the Queue.**

{

 Struct Node *tmp,*q;

 tmp = (STRUCT NODE *)malloc(sizeof(NODE));

 tmp->info = item;

 tmp->priority = priority;

if(front == NULL || priority < front->priority)

 tmp->link = front;

 front = tmp;

else

 q = front;

while(q->link != NULL && q->link->priority <= priority)

 q=q->link;

 tmp->link = q->link;

 q->link = tmp;

}

Priority Queues – Using Linked List

Delete(): It removes the highest priority element from the Queue.

```
void del()
```

```
{
```

```
    Struct Node *tmp;
```

```
    if(front == NULL)
```

```
        printf("Queue Underflow\n");
```

```
    else
```

```
    {
```

```
        tmp = front;
```

```
        printf("Deleted item is %d\n",tmp->info);
```

```
        front = front->link;
```

```
        free(tmp);
```

```
    }
```

```
}
```

Priority Queues – Using Linked List

Display()

```
{
    Struct Node *ptr;
    ptr = front;
    if(front == NULL)
        printf("Queue is empty\n");
    else
    {
        while(ptr != NULL)
        {
            printf(ptr->priority, ptr->info);
            ptr = ptr->link;
        }
    }
}
```